SQL Server Replication — Fundamentals

SQL Server Replication is a **data distribution technology** that copies and synchronizes data and schema changes from one database to one or more other databases, **near-real time or on a schedule**, while allowing different degrees of autonomy on the target systems.
At its core, replication is a **log-based change capture + agent-driven delivery system**.

1. High-Level Architecture

Replication consists of **three logical roles** and **several system components**:

Publisher → Distributor → Subscriber

Logical Flow

1. Changes occur at the **Publisher**
2. Changes are captured and stored at the **Distributor**
3. Changes are delivered and applied to the **Subscriber**

2. Core Roles (Technical View)

2.1 Publisher

What it is

- SQL Server instance hosting the **source database**
- Where data changes originate

Technical responsibilities

https://www.sqldbachamps.com/

- Maintains:
  - Replication metadata
  - Article definitions
- For transactional replication:
  - Writes changes to the **transaction log**
  - Log Reader Agent reads changes from the log

Important characteristics

- Publisher must use **FULL recovery model** (for transactional replication)
- Schema changes are tracked and optionally replicated
- Does NOT push data directly to subscribers

2.2 Distributor

What it is

- A SQL Server instance that acts as a **staging and control system**
- Hosts the **distribution database**

Key responsibility

- Stores:
  - Pending transactions
  - Replication commands
  - Agent history
  - Subscription metadata

Deployment models

- **Local Distributor** (same instance as Publisher)
- **Remote Distributor** (recommended for large systems)

Why Distributor is critical

- It decouples Publisher workload from Subscriber delivery
- Allows retries, batching, and fault tolerance

2.3 Subscriber

What it is

- SQL Server instance receiving replicated data

Capabilities

- Can be:
  - Read-only
  - Read/write (merge replication)
- May allow local changes depending on replication type

Data application

- Uses replication agents to:
  - Apply INSERT/UPDATE/DELETE
  - Optionally validate data

3. Replication Types (Internal Differences)

3.1 Snapshot Replication

Concept

- Takes a **point-in-time snapshot** of data and schema
- Applies it wholesale to subscribers

Internal Mechanics

https://www.sqldbachamps.com/

1. **Snapshot Agent**
   - Generates schema scripts
   - Exports table data to BCP files
   - Stores files in snapshot folder
2. **Distribution Agent**
   - Applies snapshot to subscriber

Technical Traits

- No change tracking
- No transaction log usage
- High I/O and network usage
- Best for:
  - Small datasets
  - Infrequent updates
  - Reference data

3.2 Transactional Replication (Most Common)

Concept

- Replicates **individual transactions** in near real-time

Internals: How Transactional Replication Works

Step 1: Change Capture (Publisher)

- SQL Server writes DML operations to the **transaction log**
- Replication marks relevant log records using:
  - **Replication Log Sequence Numbers (LSNs)**

No triggers are used. This is log-based replication.

Step 2: Log Reader Agent

Runs at Distributor

- Scans Publisher transaction log
- Identifies replicated transactions
- Converts log records into **replication commands**
- Writes commands into:
  - MSrepl_commands
  - MSrepl_transactions

📌 This step is critical:

If Log Reader Agent is blocked → replication latency grows.

Step 3: Distribution Database

Stores:

- Pending commands
- Delivery status
- Retry logic
- Agent history

This database is **write-intensive** and must be tuned carefully.

Step 4: Distribution Agent

- Reads commands from distribution database
- Delivers them to subscribers
- Applies them in the **same order** as committed at Publisher

https://www.sqldbachamps.com/

Delivery modes

- Push subscription (agent runs at Distributor)
- Pull subscription (agent runs at Subscriber)

Transaction Ordering Guarantee

- Replication preserves:
  - Transaction order
  - Commit order
- Uses LSN sequencing internally

3.3 Merge Replication

Concept

- Allows **bi-directional updates**
- Designed for:
  - Occasionally connected systems
  - Conflict detection and resolution

Technical Internals

Row Tracking

- Adds:
  - rowguid column to tables
- Uses:
  - Triggers

- Metadata tables (MSmerge_contents, MSmerge_genhistory)

Conflict Detection

- Changes are versioned
- Conflicts detected during synchronization
- Resolved using:
  - Priority rules
  - Custom resolvers

📌 Merge replication is **metadata heavy** and slower than transactional replication.

4. Replication Agents (Deep Dive)

4.1 Snapshot Agent

- Creates schema scripts
- Exports data via BCP
- Writes to snapshot folder

4.2 Log Reader Agent

- Reads transaction log
- Converts log records to replication commands
- Writes to distribution database

4.3 Distribution Agent

- Applies changes to subscriber
- Handles retries and batching
- Tracks latency and failures

4.4 Merge Agent

https://www.sqldbachamps.com/

- Synchronizes changes bi-directionally
- Detects conflicts
- Applies resolution logic

5. Articles and Publications

Publication

- A **logical container**
- Defines:
  - Replication type
  - Properties (immediate_sync, retention)

Article

- Individual object being replicated:
  - Table
  - View
  - Stored procedure
- Has:
  - Column filtering
  - Row filtering
  - Custom schema options

6. Replication Metadata (System Tables)

Publisher Database

- syspublications
- sysarticles

- syssubscriptions

Distribution Database

- MSrepl_transactions
- MSrepl_commands
- MSdistribution_history
- MSlogreader_history

Merge Metadata

- MSmerge_contents
- MSmerge_genhistory

7. Latency, Throughput, and Performance

Key Performance Factors

- Transaction log write speed
- Log Reader Agent latency
- Distribution database I/O
- Network bandwidth
- Subscriber write speed

Common Bottlenecks

- Blocking on transaction log
- Large transactions
- Long-running open transactions
- Poorly indexed subscriber tables

https://www.sqldbachamps.com/

8. Failure Scenarios & Recovery

If Subscriber Goes Down

- Commands queue up in distribution database
- Retention period applies
- Can resume automatically

If Distributor Goes Down

- Replication stops entirely
- Publisher continues logging changes
- Risk of log growth

If Retention Expires

- Subscription becomes invalid
- Requires reinitialization

9. Security Model

- SQL Agent service accounts
- Distributor admin accounts
- Snapshot folder permissions
- Linked server authentication (optional)

10. Replication vs Other Technologies

Feature	Replication	Always On	Log Shipping
Read scale	✅	✅	❌
Write scale	❌	❌	❌

Feature	Replication	Always On	Log Shipping
Cross-version	✅	❌	❌
Selective objects	✅	❌	❌
Latency	Low	Very Low	High

11. When Replication Is a Bad Choice

- High write OLTP with massive transactions
- Need for guaranteed consistency
- Complex schema changes
- Modern microservice architectures

Final Summary

SQL Server Replication is a log-driven, agent-based data movement system that:

- Captures changes from the transaction log
- Stages them in the distribution database
- Delivers them to subscribers with ordering guarantees

It is powerful, mature, and battle-tested—but requires **careful design, monitoring, and discipline**.

https://www.sqldbachamps.com/

SQL Server replication is used to **copy and synchronize data** from one database (Publisher) to another (Subscriber).
SQL Server supports **three main replication types**, each with its own use cases, advantages, and disadvantages.

1. Snapshot Replication

What it is

- Takes a **full copy (snapshot)** of the data at a specific time and applies it to subscribers.
- Does **not track individual data changes**.

Best for

- Small databases
- Data that changes infrequently
- Reporting or reference data

Advantages

✅ Simple to configure and maintain
✅ No dependency on transaction logs
✅ Reliable for static or rarely changing data

Disadvantages

❌ Copies **entire dataset each time** (slow for large databases)
❌ High network and disk usage
❌ Data can become outdated between snapshots

2. Transactional Replication

What it is

- Replicates **individual transactions** as they occur.
- Uses the **transaction log** to capture changes.

https://www.sqldbachamps.com/

Best for

- Near real-time data replication
- High-volume OLTP systems
- Read-only subscribers

Advantages

✅ Near real-time data synchronization
✅ Efficient for large databases
✅ Minimal latency
✅ Very reliable and consistent

Disadvantages

❌ More complex to configure
❌ Requires continuous connection
❌ Mostly supports **read-only subscribers**
❌ Schema changes can be tricky

3. Merge Replication

What it is

- Allows **both Publisher and Subscriber** to update data.
- Changes are merged, and conflicts are resolved using rules.

Best for

- Distributed systems
- Mobile or remote users

- Occasionally connected environments

Advantages

✅ Bi-directional updates
✅ Works offline
✅ Automatic conflict detection and resolution

Disadvantages

❌ Most complex to configure and manage
❌ Slower performance
❌ Conflict resolution can be challenging
❌ Extra overhead (triggers, metadata tables)

Comparison Table

Feature	Snapshot	Transactional	Merge
Data Updates	Full copy	Incremental	Incremental
Latency	High	Low	Medium
Subscriber Updates	No	No (mostly)	Yes
Complexity	Low	Medium	High
Best Use Case	Static data	Real-time sync	Distributed users

Quick Summary

- **Snapshot Replication** → Simple, but heavy and slow
- **Transactional Replication** → Fast and reliable for live systems
- **Merge Replication** → Flexible, but complex

https://www.sqldbachamps.com/

SQL Server Replication roles, **real-world examples**, and a **decision guide** to help choose the right replication type.

1. Replication Roles Explained

1. Publisher

What it does

- The **source database**.
- Publishes data (tables, views, procedures) to be replicated.

Key points

- Data originates here
- Tracks changes (except Snapshot)
- Can have multiple subscribers

Example

- A production database for an e-commerce website.

2. Distributor

What it does

- Acts as a **middleman**.
- Stores replication metadata and history.
- Delivers data from Publisher to Subscribers.

Key points

- Can be on the **same server** as Publisher (local distributor)
- Or on a **separate server** (remote distributor)
- Critical for performance and reliability

Example

https://www.sqldbachamps.com/

- A dedicated SQL Server instance handling replication for multiple publishers.

3. Subscriber

What it does

- Receives replicated data.
- Can be **read-only or read-write** (depends on replication type).

Key points

- Used for reporting, backups, or distributed apps
- Can subscribe to multiple publications

Example

- A reporting database used by Power BI or SSRS.

Simple Flow

Publisher → Distributor → Subscriber

2. Real-World Examples

Example 1: Reporting Database (Transactional Replication)

Scenario

- Production system with heavy transactions
- Reports slow down the main database

Solution

- Publisher: Production DB
- Subscriber: Reporting DB
- Replication Type: **Transactional**

Why

- Near real-time data
- No load on production
- Read-only reporting

Example 2: Reference Data Distribution (Snapshot Replication)

Scenario

- Country list, tax codes, or product categories
- Changes once a week/month

Solution

- Publisher: Central DB
- Subscribers: Branch databases
- Replication Type: **Snapshot**

Why

- Simple
- No need for real-time sync
- Minimal maintenance

Example 3: Mobile or Remote Offices (Merge Replication)

Scenario

- Sales staff working offline
- Data updated at HQ and branch offices

Solution

- Publisher: Head office
- Subscribers: Branch servers/laptops
- Replication Type: **Merge**

https://www.sqldbachamps.com/

Why

- Supports offline updates
- Bi-directional data changes
- Conflict resolution

Example 4: High Availability Read Scale

Scenario

- Web app needs multiple read replicas
- Writes only happen at primary DB

Solution

- Publisher: Primary DB
- Subscribers: Read replicas
- Replication Type: **Transactional**

Why

- Scales read performance
- Keeps data consistent

3. How to Choose the Best Replication Type

Quick Decision Guide

Choose Snapshot Replication if:

✓ Data changes rarely
✓ Database is small
✓ Simple setup is preferred
❌ Avoid if data is large or frequently updated

Choose Transactional Replication if:

✓ Need near real-time data
✓ High transaction volume
✓ Subscribers are read-only
✓ Reporting or scaling reads
❌ Avoid if subscribers must update data

Choose Merge Replication if:

✓ Subscribers must update data
✓ Offline work is required
✓ Distributed environments
❌ Avoid if performance and simplicity are priorities

4. Summary Table

Requirement	Best Choice
Static data	Snapshot
Real-time reporting	Transactional
Read scaling	Transactional
Offline users	Merge
Bi-directional updates	Merge
Simple setup	Snapshot

https://www.sqldbachamps.com/